Chapter 4. String Matching with Regular Expressions

4.0 Introduction

Suppose you have been on the internet for a few years and have been faithful about saving all your correspondence, just in case you (or your lawyers, or the prosecution) need a copy. The result is that you have a 5 GB disk partition dedicated to saved mail. Let's further suppose that you remember that somewhere in there is an email message from someone named Angie or Anjie. Or was it Angy? But you don't remember what you called it or where you stored it. Obviously, you have to look for it.

But while some of you might go and try to open up all 15 million documents in a word processor, I'll just find it with one simple command. Any system that provides regular expression support allows me to search for the pattern in several ways. An easy one to understand is:

```
Angie|Anjie|Angy
```

which you can probably guess means just to search for any one of the variations. A more concise form (more thinking, less typing) is:

```
An[^ dn]
```

The syntax will become clear as we go through this chapter. Briefly, the "A" and the "n" match themselves, in effect finding words that begin with "An." The cryptic `[^ dn]` requires the "An" to be followed by a character other than (^ means *not* in this context) a space (to eliminate the very common English word "an" at the start of a sentence), "d" (to eliminate the common word "and") or "n" (to eliminate "Anne," "Announcing," etc.).

Has your word processor gotten past its splash screen yet? Well, it doesn't matter, because I've already found the missing file. To find the answer, I just typed this command (it'll work on any Unix/Linux/macOS system):

```
grep 'An[^ dn]' *
```

Regular expressions, or *regexes* for short, provide a concise and precise specification of patterns to be matched in text. One good way to think of regular expressions is as a little language for matching patterns of characters in text contained in strings. A regular expression API is an interpreter for matching regular expressions.

As another example of the power of regular expressions, consider the problem of bulk-updating hundreds of files. When I started with Java, the syntax for declaring array references was `baseType arrayVariableName[]`. For example, a method with an array argument, such as every program's `main` method, was commonly written as:

```
public static void main(String args[]) {
```

But as time went by, it became clear to the stewards of the Java language that it would be better to write it as `baseType[] arrayVariableName`, like this:

```
public static void main(String[] args) {
```

This is better Java style because it associates the "array-ness" of the type with the type itself, rather than with the local argument name. While Java still accepts the old form, there is a strong preference for the new syntax.[1]

So I wanted to change all occurrences of `main` written the old way to the new way. I used the pattern `main(String [a-z]` with the `grep` utility described earlier to find the names of all the files containing old-style main declarations (i.e., `main(String` followed by a space and a name character rather than an open square bracket). I then used another regex-based Unix tool, the stream editor `sed`, in a little shell script to change all occurrences in those files from `main(String ([a-z][a-z])[]` to `main(String[] $1` (the regex syntax used

here is discussed later in this chapter). Again, the regex-based approach was orders of magnitude faster than doing it interactively, even using a reasonably powerful editor such as `vi` or `Emacs`, let alone trying to use a graphical word processor.

Historically, the syntax of regexes has changed as they get incorporated into more tools and more languages, so the exact syntax in the previous examples is not exactly what you'd use in Java, but it does convey the conciseness and power of the regex mechanism.[2]

As a third example, consider parsing an Apache web server logfile, where some fields are delimited with quotes, others with square brackets, and others with spaces. Writing ad hoc code to parse this is messy in any language, but a well-crafted regex can break the line into all its constituent fields in one operation (this example is developed in Example 4-3).

These same time gains can be had by Java developers. Regular expression support has been in the standard Java runtime for ages and is well integrated (e.g., there are regex methods in the standard class `java.lang.String` and in the no-longer-new "new I/O" package java.nio). There are a few other regex packages for Java, and you may occasionally encounter code using them, but most code from this century can be expected to use the built-in package. The syntax of Java regexes themselves is discussed in Recipe 4.1, and the syntax of the Java API for using regexes is described in Recipe 4.2. The remaining recipes show some applications of regex technology in Java.

See Also

Mastering Regular Expressions by Jeffrey Friedl (O'Reilly) is the definitive guide to all the details of regular expressions. Most introductory books on Unix and Perl include some discussion of regexes; *Unix Power Tools* by Jerry Peek, Shelley Powers, Tim O'Reilly, and Mike Loukides (O'Reilly) devotes a chapter to them.

4.1 Regular Expression Syntax

Problem

You need to learn the syntax of Java regular expressions.

Solution

Consult Table 4-1 for a list of the regular expression characters.

Discussion

These pattern characters let you specify regexes of considerable power. In building patterns, you can use any combination of ordinary text and the *metacharacters*, or special characters, in Table 4-1. These can all be used in any combination that makes sense. For example, `a+` means any number of occurrences of the letter `a`, from one up to a million or a gazillion. The pattern `Mrs?` `\.` matches `Mr.` or `Mrs.` And `.*` indicates any character, any number of times, and is similar in meaning to most command-line interpreters' meaning of the `*` alone. The pattern `\d+` means any number of numeric digits. `\d{2,3}` means a two- or three-digit number.

Table 4-1. Regular expression metacharacter syntax

Subexpression	Matches	Notes
General		
^	Start of line/string	
$	End of line/string	
\b	Word boundary	
\B	Not a word boundary	
\A	Beginning of entire string	
\z	End of entire string	
\Z	End of entire string (except allowable final line terminator)	See Recipe 4.9
.	Any one character (except line terminator)	
[...]	"Character class"; any one character from those listed	
[^...]	Any one character not from those listed	See Recipe 4.2
Alternation and grouping		
(...)	Grouping (capture groups)	See Recipe 4.4
|	Alternation	
(?:_re_)	Noncapturing parenthesis	
\G	End of the previous match	
_n	Back-reference to capture group number *n*	
Normal (greedy) quantifiers		
{*m*,*n*}	Quantifier for from *m* to *n* repetitions	See Recipe 4.5
{*m*,}	Quantifier for *m* or more repetitions	
{*m*}	Quantifier for exactly *m* repetitions	See Recipe 4.9
{,*n*}	Quantifier for 0 up to *n* repetitions	
*	Quantifier for 0 or more repetitions	Short for {0,}
+	Quantifier for 1 or more repetitions	Short for {1,}; see Recipe 4.2
?	Quantifier for 0 or 1 repetitions (i.e., present exactly once, or not at all)	Short for {0,1}
Reluctant (nongreedy) quantifiers		
{*m*,*n*}?	Reluctant quantifier for from *m* to *n* repetitions	
{*m*,}?	Reluctant quantifier for *m* or more repetitions	
{,*n*}?	Reluctant quantifier for 0 up to *n* repetitions	
*?	Reluctant quantifier: 0 or more	
+?	Reluctant quantifier: 1 or more	See Recipe 4.9
??	Reluctant quantifier: 0 or 1 times	
Possessive (very greedy)		

Subexpression	Matches	Notes
quantifiers		
`{ m , n }+`	Possessive quantifier for from *m* to *n* repetitions	
`{ m ,}+`	Possessive quantifier for *m* or more repetitions	
`{ , n }+`	Possessive quantifier for 0 up to *n* repetitions	
`*+`	Possessive quantifier: 0 or more	
`++`	Possessive quantifier: 1 or more	
`?+`	Possessive quantifier: 0 or 1 times	
Escapes and shorthands		
`\`	Escape (quote) character: turns most metacharacters off; turns subsequent alphabetic characters into metacharacters	
`\Q`	Escape (quote) all characters up to `\E`	
`\E`	Ends quoting begun with `\Q`	
`\t`	Tab character	
`\r`	Return (carriage return) character	
`\n`	Newline character	See Recipe 4.9
`\f`	Form feed	
`\w`	Character in a word	Use `\w+` for a word; see Recipe 4.9
`\W`	A nonword character	
`\d`	Numeric digit	Use `\d+` for an integer; see Recipe 4.2
`\D`	A nondigit character	
`\s`	Whitespace	Space, tab, etc., as determined by `java.lang.Character.isWhitespace()`
`\S`	A nonwhitespace character	See Recipe 4.9
Unicode blocks (representative samples)		
`\p{InGreek}`	A character in the Greek block	(Simple block)
`\P{InGreek}`	Any character not in the Greek block	
`\p{Lu}`	An uppercase letter	(Simple category)
`\p{Sc}`	A currency symbol	
POSIX-style character classes (defined only for US-ASCII)		
`\p{Alnum}`	Alphanumeric characters	`[A-Za-z0-9]`
`\p{Alpha}`	Alphabetic characters	`[A-Za-z]`
`\p{ASCII}`	Any ASCII character	`[\x00-\x7F]`
`\p{Blank}`	Space and tab characters	
`\p{Space}`	Space characters	`[ \t\n\x0B\f\r]`

Subexpression	Matches	Notes
\p{Cntrl}	Control characters	[\x00-\x1F\x7F]
\p{Digit}	Numeric digit characters	[0-9]
\p{Graph}	Printable and visible characters (not spaces or control characters)	
\p{Print}	Printable characters	Same as \p{Graph}
\p{Punct}	Punctuation characters	One of !"#$%&'()*+,-./:;<=>?@[]\^_`{|}\~
\p{Lower}	Lowercase characters	[a-z]
\p{Upper}	Uppercase characters	[A-Z]
\p{XDigit}	Hexadecimal digit characters	[0-9a-fA-F]

Regexes match any place possible in the string. Patterns followed by greedy quantifiers (the only type that existed in traditional Unix regexes) consume (match) as much as possible without compromising any subexpressions that follow. Patterns followed by possessive quantifiers match as much as possible without regard to following subexpressions. Patterns followed by reluctant quantifiers consume as few characters as possible to still get a match.

Also, unlike regex packages in some other languages, the Java regex package was designed to handle Unicode characters from the beginning. The standard Java escape sequence `\u+nnnn` is used to specify a Unicode character in the pattern. We use methods of `java.lang.Character` to determine Unicode character properties, such as whether a given character is a space.

To teach students how regexes work, I provide a little program called REDemo.[3] The code for REDemo is too long to include in the book; in the online directory *regex* of the *darwinsys-api* repo, you will find *REDemo.java*, which can be run to explore how regexes work. It's also available online at my GitHub repository.

In the uppermost text box (see Figure 4-1), type the regex pattern you want to test. As you type each character, the regex is checked for syntax; if the syntax is OK, you see a checkmark beside it. You can then select Match, Find, or Find All. Match means that the entire string must match the regex, and Find means the regex must be found somewhere in the string. (Find All counts the number of occurrences that are found.) Below that, you type a string that the regex is to match against. Experiment to your heart's content. When you have the regex the way you want it, you can paste it into your Java program. You'll need to escape (backslash) any characters that are treated specially by both the Java compiler and the Java regex package, such as the backslash itself, double quotes, and others. Once you get a regex the way you want it, there is a Copy button (not shown in these screenshots) to export the regex to the clipboard, with or without backslash doubling, depending on how you want to use it.

TIP

Remember that because a regex is entered as a string that will be compiled by a Java compiler, you usually need two levels of escaping for any special characters, including backslash and double quotes. For example, the regex (which includes the double quotes):

```
"You said it\."
```

has to be typed like this to be a valid compile-time Java language `String`:

```
String pattern = "\"You said it\\.\"";
```

In Java 14+ you can also use a `String` text block to avoid escaping the quotes:

```
String pattern = """
    "You said it\\."""";
```

I can't tell you how many times I've made the mistake of forgetting the extra backslash in `\d+`, `\w+`, and their kin!

In Figure 4-1, I typed `qu` into the REDemo program's Pattern box, which is a syntactically valid regex pattern: any ordinary characters stand as regexes for themselves, so this looks for the letter `q` followed by `u`. In the top version, I typed only a `q` into the string, and

this is not matched. In the second, I have typed `quack` and the `q` of a second `quack`. Because I have selected Find All, the count shows one match. As soon as I type the second `u`, the count is updated to two, as shown in the third version.

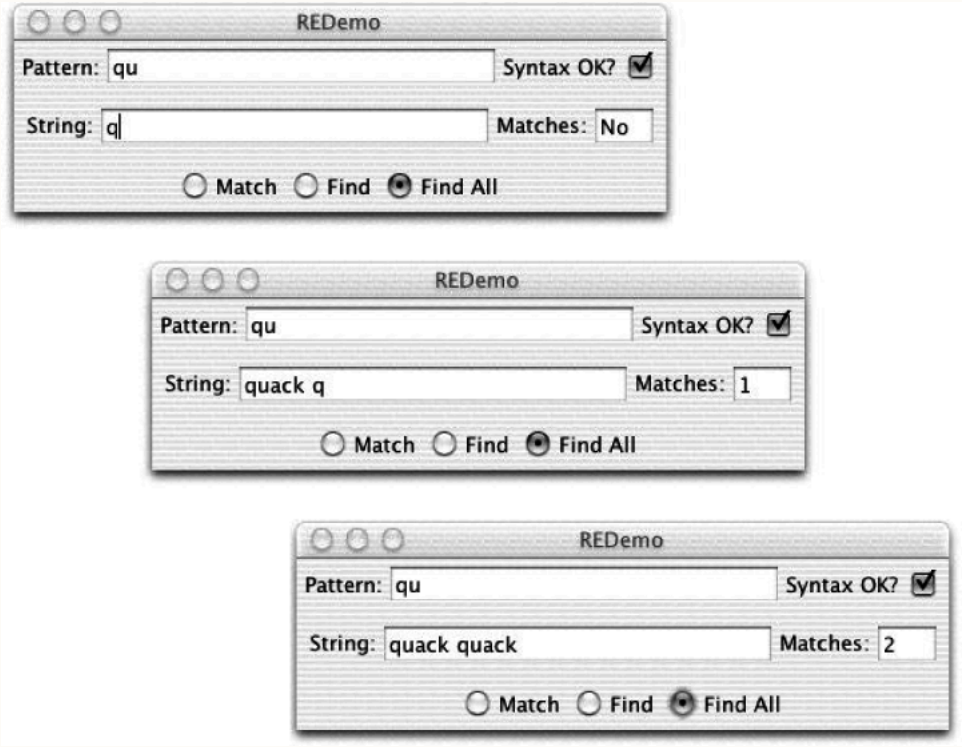


Figure 4-1. REDemo with simple examples

Regexes can do far more than just character matching. For example, the two-character regex `^T` would match beginning of line (`^`) immediately followed by a capital T—that is, any line beginning with a capital T. It doesn't matter whether the line begins with "Tiny trumpets," "Titanic tubas," or "Triumphant twisted trombones," as long as the capital T is present in the first position.

But here we're not very far ahead. Have we really invested all this effort in regex technology just to be able to do what we could already do with the `java.lang.String` method `startsWith()`? Hmmm, I can hear some of you getting a bit restless. Stay in your seats! What if you wanted to match not only a letter T in the first position, but also a vowel immediately after it, followed by any number of letters in a word, followed by an exclamation point? Surely you could do this in Java by checking `startsWith("T")` and `charAt(1) == 'a' || charAt(1) == 'e'`, and so on? Yes, but by the time you did that, you'd have written a lot of very highly specialized code that you couldn't use in any other application. With regular expressions, you can just give the pattern `^T[aeiou]\w*!`. That is, `^` and `T` as before, followed by a character class listing the vowels, followed by any number of word characters (`\w*`), followed by the exclamation point.

"But wait, there's more!" as my late, great boss Yuri Rubinsky used to say. What if you want to be able to change the pattern you're looking for *at runtime*? Remember all that Java code you just wrote to match `T` in column 1, plus a vowel, some word characters, and an exclamation point? Well, it's time to throw it out. Because this morning we need to match `Q`, followed by a letter other than `u`, followed by a number of digits, followed by a period. While some of you start writing a new function to do that, the rest of us will just saunter over to the RegEx Bar & Grille, order a `^Q[^u]\d+\.` from the bartender, and be on our way.

OK, if you want an explanation: the `[^u]` means match any one character that is not the character `u`. The `\d+` means one or more numeric digits. The `+` is a quantifier meaning one or more occurrences of what it follows, and `\d` is any one numeric digit. So `\d+` means a number with one, two, or more digits. Finally, the `\.`? Well, . by itself is a metacharacter. Most single metacharacters are switched off by preceding them with an escape character. Not the Esc key on your keyboard, of course. The regex escape character is the backslash. Preceding a metacharacter like . with this escape turns off its special meaning, so we look for a literal period rather than any character. Preceding a few selected alphabetic characters (e.g., `n`, `r`, `t`, `s`, `w`) with escape turns them into metacharacters.

Figure 4-2 shows the `^Q[^u]\d+\.` regex in action. In the first frame, I have typed part of the regex as `^Q[^u`. Because there is an unclosed square bracket, the Syntax OK flag is turned off; when I complete the regex, it will be turned back on. In the second frame, I have finished typing the regex, and I've typed the data string as `QA577` (which you should expect to match the `^Q[^u]\\d+` but not the period since I haven't typed it). In the third frame, I've typed the period, so the Matches flag is set to Yes.

Because backslashes need to be escaped when pasting the regex into Java code, the current version of REDemo has both a Copy Pattern button, which copies the regex verbatim for use in documentation and in Unix commands, and a Copy Pattern Backslashed button, which copies the regex to the clipboard with backslashes doubled, for pasting into Java strings.

By now you should have at least a basic grasp of how regexes work in practice. The rest of this chapter gives more examples and explains some of the more powerful topics, such as capture groups. As for how regexes work in theory—and there are a lot of theoretical details and differences among regex flavors—the interested reader is referred to *Mastering Regular Expressions*. Meanwhile, let's start learning how to write Java programs that use regular expressions.

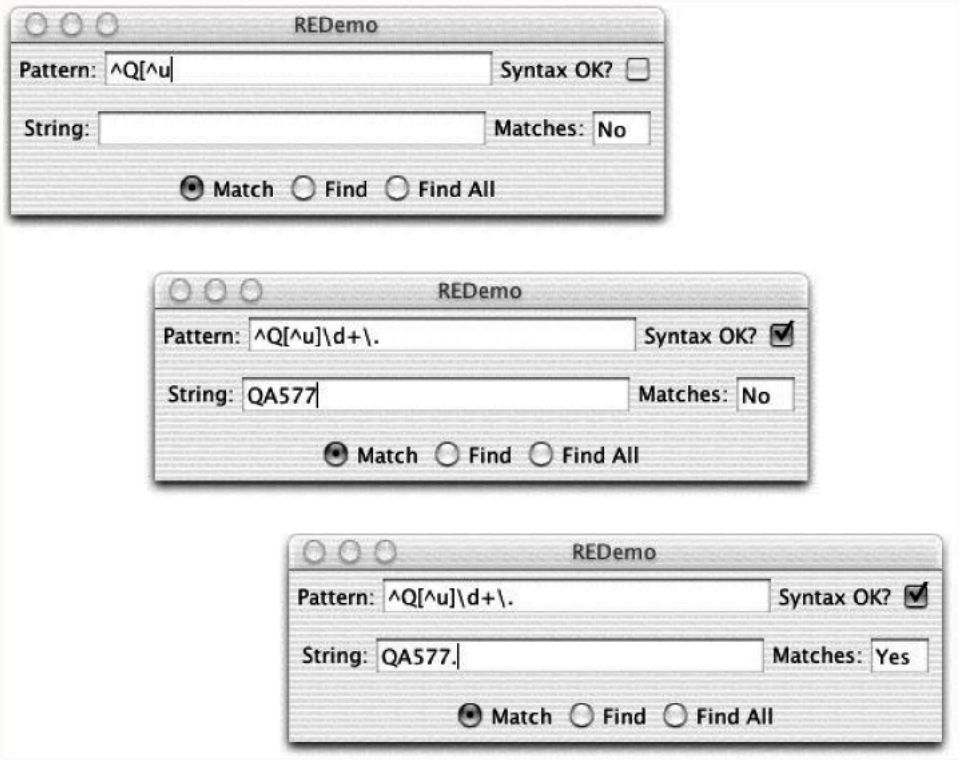


Figure 4-2. REDemo with "Q not followed by u" example

4.2 Checking If a String Matches a Regex

Problem

You're ready to get started using regular expression processing to beef up your Java code by testing to see if a given pattern can match in a given string.

Solution

Use the Java regular expressions package, java.util.regex.

Discussion

The good news is that the Java API for regexes is actually easy to use. If all you need is to find out whether a given regex matches a string, you can use the convenient `boolean matches()` method of the `String` class, which accepts a regex pattern in `String` form as its argument:

```
if (inputString.matches(stringRegexPattern)) {
   // it matched... do something with it...
}
```

This is, however, a convenience routine, and convenience always comes at a price. If the regex is going to be used more than once or twice in a program, it is more efficient to construct and use a `Pattern` and its `Matcher`(s). A complete program constructing a `Pattern` and using it to match against strings is shown in Example 4-1.

Example 4-1. main/src/main/java/regex/RESimple.java

```
public class RESimple {
  public static void main(String[] argv) {
    String pattern = "^Q[^u]\\d+\\.";
    String[] input = {
```

```
    "QA777. is the next flight. It is on time.",
    "Quack, Quack, Quack!"
   };

   Pattern p = Pattern.compile(pattern);

   for (String in : input) {
    boolean found = p.matcher(in).lookingAt();

    System.out.println("" + pattern + "" +
    (found ? " matches '" : " doesn't match '") + in + "'");
   }
  }
}
```

The java.util.regex package contains two classes, `Pattern` and `Matcher`, which provide the public API shown in Example 4-2.

Example 4-2. Regex public API

```
/**
 * The main public API of the java.util.regex package.
 */
package java.util.regex;

public final class Pattern {
  // Flags values ('or' together)
  public static final int
    UNIX_LINES, CASE_INSENSITIVE, COMMENTS, MULTILINE,
    DOTALL, UNICODE_CASE, CANON_EQ;
  // No public constructors; use these Factory methods
  public static Pattern compile(String patt);
  public static Pattern compile(String patt, int flags);
  // Method to get a Matcher for this Pattern
  public Matcher matcher(CharSequence input);
  // Information methods
  public String pattern();
  public int flags();
  // Convenience methods
  public static boolean matches(String pattern, CharSequence input);
  public String[] split(CharSequence input);
  public String[] split(CharSequence input, int max);
}

public final class Matcher {
  // Action: find or match methods
  public boolean matches();
  public boolean find();
  public boolean find(int start);
  public boolean lookingAt();
  // "Information about the previous match" methods
  public int start();
  public int start(int whichGroup);
  public int end();
  public int end(int whichGroup);
  public int groupCount();
  public String group();
  public String group(int whichGroup);
  // Reset methods
  public Matcher reset();
  public Matcher reset(CharSequence newInput);
  // Replacement methods
  public Matcher appendReplacement(StringBuffer where, String newText);
  public StringBuffer appendTail(StringBuffer where);
  public String replaceAll(String newText);
  public String replaceFirst(String newText);
  // information methods
  public Pattern pattern();
}
```

```
/* String, showing only the RE-related methods */
public final class java.lang.String {
  public boolean matches(String regex);
  public String replaceFirst(String regex, String newStr);
  public String replaceAll(String regex, String newStr);
  public String[] split(String regex);
  public String[] split(String regex, int max);
  ...
}
```

This API is large enough to require some explanation. These are the typical steps for regex matching in a production program:

1. Create a `Pattern` by calling the static method `Pattern.compile()`.
2. Request a `Matcher` from the `Pattern` by calling `pattern.matcher(CharSequence)` for each `String` (or other `CharSequence`) you wish to look through.
3. Call (once or more) one of the finder methods (discussed later in this section) in the resulting `Matcher`.

The `java.lang.CharSequence` interface provides simple read-only access to objects containing a collection of characters. The standard implementations are `String` and `StringBuffer` / `StringBuilder` (described in Chapter 3), and the new I/O class `java.nio.CharBuffer`.

Of course, you can perform regex matching in other ways, such as using the convenience methods in `Pattern` or even in `java.lang.String`, like this:

```
public class StringConvenience {
  public static void main(String[] argv) {

    String pattern = ".*Q[^u]\\d+\\..*";
    String line = "Order QT300. Now!";
    if (line.matches(pattern)) {
      System.out.println(line + " matches \"" + pattern + "\"");
    } else {
      System.out.println("NO MATCH");
    }
  }
}
```

But the three-step list is the standard pattern for matching. You'd likely use the `String` convenience routine in a program that only used the regex once; if the regex were being used more than once, it is worth taking the time to compile it because the compiled version runs faster.

In addition, the `Matcher` has several finder methods, which provide more flexibility than the `String` convenience routine `match()`. These are the `Matcher` methods:

`match()`

Used to compare the entire string against the pattern; this is the same as the routine in `java.lang.String`. Because it matches the entire `String`, I had to put `.*` before and after the pattern.

`lookingAt()`

Used to match the pattern only at the beginning of the string.

`find()`

Used to match the pattern in the string (not necessarily at the first character of the string), starting at the beginning of the string or, if the method was previously called and succeeded, at the first character not matched by the previous match.

Each of these methods returns `boolean`, with `true` meaning a match and `false` meaning no match. To check whether a given string matches a given pattern, you need only type something like the following:

```
Matcher m = Pattern.compile(patt).matcher(line);
if (m.find( )) {
```

```
    System.out.println(line + " matches " + patt)
}
```

When constructing a `Pattern` whose regex syntax is complex, you can use the multi-line or commented style by passing `Pattern.COMMENTS` as the second argument to the `compile()` method:

```
Pattern.compile("""
      \s*  # leading space
      \d+  # number
      \w+ # name
""", Pattern.COMMENTS);
```

It's recommended to use the `String` text block style as shown (discussed near the end of the introduction to Chapter 3), and it's important to *not* put commas after each component. A longer example is shown in the following recipe, Recipe 4.3.

You may want more flexibility to extract the part of text that matched. This is the subject of the next recipes, which cover uses of the Matcher API. Initially, the examples just use arguments of type `String` as the input source. Use of other `CharSequence` types is covered in Recipe 4.6.

4.3 Grouping: Specifying Parts of the Regex

Problem

You need to refer to text that matched part of the regex, rather than the text that matched the entire regex.

Solution

Use parenthesized groups to delimit the subpart of the regex. Use the `Matcher` method `group()` to refer to the corresponding submatch.

Discussion

Using groups to enable access to part of a regex is a fundamental part of regex work. Suppose we need to parse text like a logfile, which has a number of fields or columns. We want to print out individual fields from the text. Example 4-3 demonstrates how to do this.

Example 4-3. main/src/main/java/regex/LogRegEx.java—Apache Log File Scanner

```
public class LogRegEx {

  public static final int MIN_FIELDS = 8;

  final static String SAMPLE_LINE = "123.45.67.89 - - [27/Oct/2000:09:27:09 -0400] \"GET /java/javaResources.html HTTP/1.0\" 200 10450 \"-\" \"Moz

  final static String LOG_ENTRY_PATTERN = """
    ^([\\w\\d.-]+)\\s+              # 1 - IP
    (\\S+)\\s+                    # 2 - User, from identd (always "-")
    (\\S+)\\s+                    # 3 - User, from https (often "-")
    \\[([\\w:/]+\\s[+-]\\d{4})\\]\\s+ # 4 - Date, time, space, timezone-offset
    ([a-zA-Z.]+\\s+)?               # 5 - domainname, in some formats
    "(.+?)"\\s+                   # 6 - Entire request line
    (\\d{3})\\s+                   # 7 - Status code (200, 404, etc)
    (\\d+)\\s*                    # 8 - Byte count
    ("[^"]+"\\s*)?                  # 9 - Referrer, or "-"
    ("([^"]+)")?                  # 10 - Browser advertising clause, free-form
    """;

  final static Pattern PATT = Pattern.compile(LOG_ENTRY_PATTERN, Pattern.COMMENTS);

  public static void main(String argv[]) throws IOException {
```

```
    System.out.println("RE Pattern:");
    System.out.println(PATT);

    if (argv.length == 0) {
      process(SAMPLE_LINE);
    } else {
      for (String fileName : argv) {
        Files.lines(Path.of(fileName)).forEach(LogRegEx::process);
      }
    }
  }

  static void process(String logEntryLine) {

    System.out.println("Input line:" + logEntryLine);
    Matcher matcher = PATT.matcher(logEntryLine);
    if (!matcher.find()) {
      System.err.println("Failed to match (Bad log entry or problem with regex)");
      return;
    }
    if (matcher.groupCount() < MIN_FIELDS) {
      System.err.println("Matched, but has too few fields):");
      return;
    }
    System.out.println("IP Address: " + matcher.group(1));
    System.out.println("UserName: " + matcher.group(3));
    System.out.println("Date/Time: " + matcher.group(4));
    System.out.println("Request: " + matcher.group(5));
    System.out.println("Response: " + matcher.group(7));
    System.out.println("Byte Count: " + matcher.group(8));
    if (!matcher.group(9).equals("-"))
      System.out.println("Referer: " + matcher.group(9));
    System.out.println("User-Agent: " + matcher.group(10));
  }
}
```

4.4 Finding the Matching Text

Problem

You need to find the text that the regex matched.

Solution

Use the methods of the `Matcher` class.

Discussion

Sometimes you need to know more than just whether a regex matched a string. In editors and many other tools, you want to know exactly which characters were matched. Remember that with quantifiers such as *, the length of the text that was matched may have no relationship to the length of the pattern that matched it. Do not underestimate the mighty .*, which happily matches thousands or millions of characters if allowed to. As you saw in the previous recipe, you can find out whether a given match succeeds just by using `find()` or `matches()`. But in other applications, you will want to get the characters that the pattern matched.

After a successful call to one of the preceding methods, you can use these information methods on the `Matcher` to get information on the match:

`start(), end()`

Returns the character position in the string of the starting and ending characters that matched.

`groupCount()`

Returns the number of parenthesized capture groups, if any; returns 0 if no groups were used.

group(int i)

Returns the characters matched by group *i* of the current match, if *i* is greater than or equal to zero and less than or equal to the return value of `groupCount()`. Group 0 is the entire match, so `group(0)` (or just `group()`) returns the entire portion of the input that matched.

The notion of parentheses, or capture groups, is central to regex processing. Regexes may be nested to any level of complexity. The `group(int)` method lets you retrieve the characters that matched a given parenthesis group. If you haven't used any explicit parens, you can just treat whatever matched as level zero. Example 4-4 shows part of *REMatch.java*.

Example 4-4. Part of main/src/main/java/regex/REMatch.java

```
public class REmatch {
 public static void main(String[] argv) {

   String patt = "Q[^u]\\d+\\.";
   Pattern r = Pattern.compile(patt);
   String line = "Order QT300. Now!";
   Matcher m = r.matcher(line);
   if (m.find()) {
    System.out.println(patt + " matches \"" +
      m.group(0) +
      "\" in \"" + line + "\"");
   } else {
    System.out.println("NO MATCH");
   }
  }
}
```

When run, this prints:

```
Q[^u]\d+\. matches "QT300." in "Order QT300. Now!"
```

With the `Match` button checked, REDemo provides a display of all the capture groups in a given regex; one example is shown in Figure 4-3.

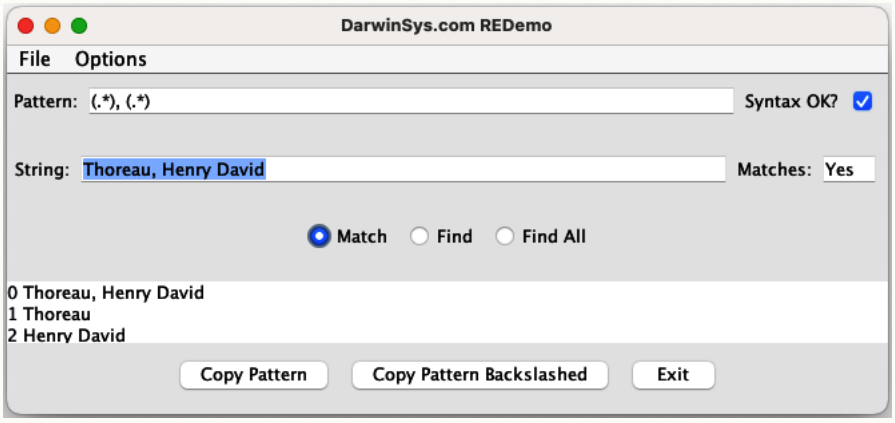


Figure 4-3. REDemo in action

It is also possible to get the starting and ending indices and the length of the text that the pattern matched (remember that terms with quantifiers, such as the `\d+` in this example, can match an arbitrary number of characters in the string). You can use these in conjunction with the `String.substring()` methods as follows:

```
   String patt = "Q[^u]\\d+\\.";
   Pattern r = Pattern.compile(patt);
   String line = "Order QT300. Now!";
   Matcher m = r.matcher(line);
   if (m.find()) {
    System.out.println(patt + " matches \"" +
      line.substring(m.start(0), m.end(0)) +
      "\" in \"" + line + "\"");
   } else {
```

```
    System.out.println("NO MATCH");
  }
```

Suppose you need to extract several items from a string. If the input is:

```
Smith, John
Adams, John Quincy
```

and you want to get out:

```
John Smith
John Quincy Adams
```

just use the code in Example 4-5.

Example 4-5. main/src/main/java/regex/REmatchTwoFields.java

```
public class REmatchTwoFields {
 public static void main(String[] args) {
  String inputLine = "Adams, John Quincy";
  // Construct an RE with parens to "grab" last name and first name
  Pattern p = Pattern.compile("(.*), (.*)");
  Matcher m = p.matcher(inputLine);
  if (!m.matches()) {
   throw new IllegalArgumentException("Bad input");
  }
  System.out.println("Numbered: " + m.group(2) + ' ' + m.group(1));

  // Same thing but with names:
  Pattern p2 = Pattern.compile("(?<last>.*), (?<first>.*)");
  m = p2.matcher(inputLine);
  if (!m.matches()) {
   throw new IllegalArgumentException("Bad input");
  }
  System.out.println("Named 1:  " + m.group("first") + " " + m.group("last"));
  System.out.println("Named 2:  " + m.replaceAll("${first} ${last}"));
 }
}
```

While numbered groups are fine when there's only a few, as the regex grows more complex, it makes sense to use named groups, since adding or removing one renumbers all those after it. While numbered groups are entered using `(subPattern)`, named groups are entered using `(?<_name_>subPattern)`, as in the second part of Example 4-5. These can be used in calls to `matcher.group("`*name*`")` or using `${`*name*`}` in the `matcher.replace...` methods.

4.5 Replacing the Matched Text

Problem

Having found some text using a `Pattern`, you want to replace the text with different text, without disturbing the rest of the string.

Solution

As we saw in the previous recipe, regex patterns involving quantifiers can match a lot of characters with very few metacharacters. We need a way to replace the text that the regex matched without changing other text before or after it. We could do this manually using the `String` method `substring()`. However, because it's such a common requirement, the Java Regular Expression API provides some substitution methods.

Discussion

The `Matcher` class provides several methods for replacing just the text that matched the pattern. In all these methods, you pass in the replacement text, or "righthand side" of the substitution. (This term is historical: in a command-line text editor's substitute command, the lefthand side is the pattern and the righthand side is the replacement text.) These are the replacement methods:

`replaceAll(newString)`

Replaces all occurrences that matched with the new string

`replaceFirst(newString)`

Similar to `replaceAll(newString)` but only replaces the first occurence

`appendReplacement(StringBuffer, newString)`

Copies up to before the first match, plus the given `newString`

`appendTail(StringBuffer)`

Appends text after the last match (normally used after `appendReplacement`)

Despite their names, the `replace*` methods behave in accordance with the immutability of `Strings` (see "Timeless, Immutable, and Unchangeable"): they create a new `String` object with the replacement performed. They do not (indeed, could not) modify the string referred to in the `Matcher` object.

Example 4-6 shows use of these three methods.

Example 4-6. main/src/main/java/regex/ReplaceDemo.java

```
/**
 * Quick demo of RE substitution: correct U.S. 'favor'
 * to Canadian/British 'favour', but not in "favorite"
 */
public class ReplaceDemo {
  public static void main(String[] argv) {

    // Make an RE pattern to match as a word only (\b=word boundary)
    String patt = "\\bfavor\\b";

    // A test input.
    String input = "Do me a favor? Fetch my favorite.";
    System.out.println("Input: " + input);

    // Run it from an RE instance and see that it works
    Pattern r = Pattern.compile(patt);
    Matcher m = r.matcher(input);
    System.out.println("ReplaceAll: " + m.replaceAll("favour"));

    // Show the appendReplacement method
    m.reset();
    StringBuilder sb = new StringBuilder();
    System.out.print("Append methods: ");
    while (m.find()) {
      // Copy to before first match,
      // plus the word "favor"
      m.appendReplacement(sb, "favour");
    }
    m.appendTail(sb);    // copy remainder
    System.out.println(sb.toString());
  }
}
```

Sure enough, when you run it, it does what we expect:

```
Input: Do me a favor? Fetch my favorite.
ReplaceAll: Do me a favour? Fetch my favorite.
```

Append methods: Do me a favour? Fetch my favorite.

The `replaceAll()` method handles the case of making the same change all through a string. If you want to change each matching occurrence to a different value, you can use `replaceFirst()` in a loop, as in Example 4-7. Here we make a pass through an entire string, turning each occurrence of either `cat` or `dog` into `feline` or `canine`. This is simplified from a real example that looked for *bit.ly* URLs and replaced them with the actual URL; the `computeReplacement` method there used the network client code from Recipe 14.1.

Example 4-7. main/src/main/java/regex/ReplaceMulti.java

```
/**
 * To perform multiple distinct substitutions in the same String,
 * you need a loop, and must call reset() on the matcher.
 */
public class ReplaceMulti {
  public static void main(String[] args) {

    Pattern patt = Pattern.compile("cat|dog");
    String line = "The cat and the dog never got along well.";
    System.out.println("Input: " + line);
    Matcher matcher = patt.matcher(line);
    while (matcher.find()) {
      String found = matcher.group(0);
      String replacement = computeReplacement(found);
      line = matcher.replaceFirst(replacement);
      matcher.reset(line);
    }
    System.out.println("Final: " + line);
  }

  static String computeReplacement(String in) {
    switch(in) {
    case "cat": return "feline";
    case "dog": return "canine";
    default: return "animal";
    }
  }
}
```

If you need to refer to portions of the occurrence that matched the regex, you can mark them with extra parentheses in the pattern and refer to the matching portion with `$1`, `$2`, and so on in the replacement string. Example 4-8 uses this to interchange two fields, in this case, turn names in the form `Firstname Lastname` into `Lastname, FirstName`.

Example 4-8. main/src/main/java/regex/ReplaceDemo2.java

```
public class ReplaceDemo2 {
  public static void main(String[] argv) {

    // Make an RE pattern
    String patt = "(\\w+)\\s+(\\w+)";

    // A test input.
    String input = "Ian Darwin";
    System.out.println("Input: " + input);

    // Run it from an RE instance and see that it works
    Pattern r = Pattern.compile(patt);
    Matcher m = r.matcher(input);
    m.find();
    System.out.println("Replaced: " + m.replaceFirst("$2, $1"));

    // The short inline version:
    // System.out.println(input.replaceFirst("(\\w+)\\s+(\\w+)", "$2, $1"));
  }
}
```

4.6 Printing All Occurrences of a Pattern

Problem

You need to find all the strings that match a given regex in one or more files or other sources.

Solution

Compare each line against the regex pattern.

Discussion

This example reads through a file one line at a time. Whenever a match is found, I extract it from the `line` and print it.

This code takes the `group()` methods from Recipe 4.4, the `substring` method from the `CharacterIterator` interface, and the `match()` method from the regex and simply puts them all together. I coded it to extract all the names from a given file; in running the program through itself, it prints the words `import`, `java`, `until`, `regex`, and so on, each on its own line:

```
C:\> java ReaderIter.java ReaderIter.java
import
java
util
regex
import
java
io
Print
all
the
strings
that
match
given
pattern
from
file
public
...
C:\\>
```

I interrupted it here to save paper. This can be written two ways: a line-at-a-time pattern shown in Example 4-9 and a more efficient form using new I/O shown in Example 4-10 (the new I/O package used in both examples is described in Chapter 10).

Example 4-9. main/src/main/java/regex/ReaderIter.java

```
public class ReaderIter {
 public static void main(String[] args) throws IOException {
  // The RE pattern
  Pattern patt = Pattern.compile("[A-Za-z][a-z]+");
  // See the I/O chapter
  // For each line of input, try matching in it.
  Files.lines(Path.of(args[0])).forEach(line -> {
   // For each match in the line, extract and print it.
   Matcher m = patt.matcher(line);
   while (m.find()) {
    // Simplest method:
    // System.out.println(m.group(0));

    // Get the starting position of the text
    int start = m.start(0);
    // Get ending position
    int end = m.end(0);
    // Print whatever matched.
    // Use CharacterIterator.substring(offset, end);
```

```
        System.out.println(line.substring(start, end));
      }
    });
  }
}
```

Example 4-10. main/src/main/java/regex/GrepNIO.java

```
public class GrepNIO {
  public static void main(String[] args) throws IOException {

    if (args.length < 2) {
      System.err.println("Usage: GrepNIO patt file [...]");
      System.exit(1);
    }

    Pattern p=Pattern.compile(args[0]);
    for (int i=1; i<args.length; i++)
      process(p, args[i]);
  }

  static void process(Pattern pattern, String fileName) throws IOException {

    // Get a FileChannel from the given file.
    FileInputStream fis = new FileInputStream(fileName);
    FileChannel fc = fis.getChannel();

    // Map the file's content
    ByteBuffer buf = fc.map(FileChannel.MapMode.READ_ONLY, 0, fc.size());

    // Decode ByteBuffer into CharBuffer
    CharBuffer cbuf =
      Charset.forName("ISO-8859-1").newDecoder().decode(buf);

    Matcher m = pattern.matcher(cbuf);
    while (m.find()) {
      System.out.println(m.group(0));
    }
    fis.close();
  }
}
```

The nonblocking I/O version shown in Example 4-10 uses the new I/O (NIO) package java.nio, and relies on the fact that a nonblocking I/O `Buffer` can be used as a `CharSequence`. This program is more general in that the pattern argument is taken from the command-line argument. It prints the same output as the previous example if invoked with the pattern argument from the previous program on the command line:

```
java regex.GrepNIO "[A-Za-z][a-z]+"  ReaderIter.java
```

You might think of using `\w+` as the pattern; the only difference is that my pattern looks for well-formed capitalized words, whereas `\w+` would include Java-centric oddities like `theVariableName`, which have capitals in nonstandard positions.

Also note that the NIO version will probably be more efficient because it doesn't reset the `Matcher` to a new input source on each line of input as `ReaderIter` does.

4.7 Controlling Case in Regular Expressions

Problem

You want to find text regardless of case.

Solution

Use the `CASE_INSENSITIVE` option of the `Pattern.compile()` method.

Discussion

Compile the `Pattern` passing in the `flags` argument `Pattern.CASE_INSENSITIVE` to indicate that matching should be case-independent (i.e., that it should fold, ignore differences in case). If your code might run in different locales (see Recipe 3.12), then you should add `Pattern.UNICODE_CASE`. Without these flags, the default is normal, case-sensitive matching behavior. This flag (and others) is passed to the `Pattern.compile()` method, like this:

```
// regex/CaseMatch.java
Pattern  reCaseInsens = Pattern.compile(pattern, Pattern.CASE_INSENSITIVE |
   Pattern.UNICODE_CASE);
reCaseInsens.matches(input);       // will match case-insensitively
```

This flag must be passed when you create the `Pattern`; because `Pattern` objects are immutable, they cannot be changed once constructed.

The full source code for this example is online as *CaseMatch.java*.

PATTERN.COMPILE() FLAGS

Half a dozen flags can be passed as the second argument to `Pattern.compile()`. If more than one value is needed, they can be or'd together using the bitwise or operator `|`. In alphabetical order, these are the flags:

CANON_EQ

Enables so-called canonical equivalence. In other words, characters are matched by their base character so that the character `e` followed by the combining character mark for the acute accent (´) can be matched either by the composite character `é` or the letter `e` followed by the character mark for the accent (see Recipe 4.8).

CASE_INSENSITIVE

Turns on case-insensitive matching (see Recipe 4.7).

COMMENTS

Causes whitespace and comments (from `#` to end-of-line) to be ignored in the pattern. See *CommentedRegEx.java* in the *regex* source directory.

DOTALL

Allows dot (.) to match any regular character or the newline, not just any regular character other than newline (see Recipe 4.9).

MULTILINE

Specifies multiline mode (see Recipe 4.9).

UNICODE_CASE

Enables Unicode-aware case folding (see Recipe 4.7).

UNIX_LINES

Makes `\n` the only valid newline sequence for `MULTILINE` mode (see Recipe 4.9).

4.8 Matching Accented, or Composite, Characters

Problem

You want characters to match regardless of the form in which they are entered.

Solution

Compile the `Pattern` with the `flags` argument `Pattern.CANON_EQ` for canonical equality.

Discussion

Composite characters can be entered in various forms. Consider, as a single example, the letter `e` with an acute accent. This character can be found in various forms in Unicode text, such as the single character `é` (Unicode character `\u00e9`) or the two-character sequence `e´` (e followed by the Unicode combining acute accent, `\u0301`). To allow you to match such characters regardless of which of possibly multiple fully decomposed forms are used to enter them, the regex package has an option for *canonical matching*, which treats any of the forms as equivalent. This option is enabled by passing `CANON_EQ` as (one of) the flags in the second argument to `Pattern.compile()`. Example 4-11 shows `CANON_EQ` being used to match several forms.

Example 4-11. main/src/main/java/regex/CanonEqDemo.java

```
public class CanonEqDemo {
  public static void main(String[] args) {
    String pattStr = "\u00e9gal"; // egal
    String[] input = {
        "\u00e9gal", // egal - this one had better match :-)
        "e\u0301gal", // e + "Combining acute accent"
        "e\u02cagal", // e + "modifier letter acute accent"
        "e'gal", // e + single quote
        "e\u00b4gal", // e + Latin-1 "acute"
    };
    Pattern pattern = Pattern.compile(pattStr, Pattern.CANON_EQ);
    for (int i = 0; i < input.length; i++) {
      if (pattern.matcher(input[i]).matches()) {
        System.out.println(pattStr + " matches input " + input[i]);
      } else {
        System.out.println(pattStr + " doesn't match input " + input[i]);
      }
    }
  }
}
```

This program correctly matches the combining accent and rejects the other characters, some of which, unfortunately, look like the accent on a printer, but are not considered combining accent characters:

```
égal matches input égal
égal matches input e?gal
égal does not match input e?gal
égal does not match input e'gal
égal does not match input e´gal
```

For more details, see the Unicode character charts.

4.9 Matching Newlines in Text

Problem

You need to match newlines in text.

Solution

Use `\n` or `\r` in your regex pattern. See also the flags constant `Pattern.MULTILINE`, which makes newlines match as beginning-of-line and end-of-line (`^` and `$`).

Discussion

Though line-oriented tools from Unix such as `sed` and `grep` match regular expressions one line at a time, not all tools do. The `Sam` text editor from Bell Laboratories was the first interactive tool I know of to allow multiline regular expressions; the Perl scripting language followed shortly after. In the Java API, the newline character by default has no special significance. The `BufferedReader` method `readLine()` normally strips out whichever newline characters it finds. If you read in gobs of characters using some method other than `readLine()`, you may have some number of `\n`, `\r`, or `\r\n` sequences in your text string.[4] Normally all of these are treated as equivalent to `\n`. If you want only `\n` to match, use the `UNIX_LINES` flag to the `Pattern.compile()` method.

In Unix, `^` and `$` are commonly used to match the beginning or end of a line, respectively. In this API, the regex metacharacters `^` and `$` ignore line terminators and only match at the beginning and the end, respectively, of the entire string. However, if you pass the `MULTILINE` flag into `Pattern.compile()`, these expressions match just after or just before, respectively, a line terminator; `$` also matches the very end of the string. Because the line ending is just an ordinary character, you can match it with . or similar expressions; and, if you want to know exactly where it is, `\n` or `\r` in the pattern match it as well. In other words, to this API, a newline character is just another character with no special significance (see the sidebar "Pattern.compile() Flags"). An example of newline matching is shown in Example 4-12.

Example 4-12. main/src/main/java/regex/NLMatch.java

```
public class NLMatch {
  public static void main(String[] argv) {

    String input = "I dream of engines\nmore engines, all day long";
    System.out.println("INPUT: " + input);
    System.out.println();

    String[] patt = {
      "engines.more engines",
      "ines\nmore",
      "engines$"
    };

    for (int i = 0; i < patt.length; i++) {
      System.out.println("PATTERN " + patt[i]);

      boolean found;
      Pattern p1l = Pattern.compile(patt[i]);
      found = p1l.matcher(input).find();
      System.out.println("DEFAULT match " + found);

      Pattern pml = Pattern.compile(patt[i],
        Pattern.DOTALL|Pattern.MULTILINE);
      found = pml.matcher(input).find();
      System.out.println("MultiLine match " + found);
      System.out.println();
    }
  }
}
```

If you run this code, the first pattern (with the wildcard character .) always matches, whereas the second pattern (with `$`) matches only when `MATCH_MULTILINE` is set:

```
> java regex.NLMatch
INPUT: I dream of engines
more engines, all day long

PATTERN engines
more engines
DEFAULT match true
MULTILINE match: true

PATTERN engines$
DEFAULT match false
MULTILINE match: true
```

4.10 Program: Full Grep

Now that we've seen how the regular expressions package works, it's time to write JGrep, a full-blown version of the line-matching program with option parsing. Table 4-2 lists some typical command-line options that a Unix implementation of `grep` might include. For those not familiar with `grep`, it is a command-line tool that searches for regular expressions in text files. There are three or four programs in the standard `grep` family, and several newer replacements such as `ripgrep`, or `rg`. This program is my addition to this family of programs.

Table 4-2. `Grep` command-line options

Option	Meaning
`-c`	Count only; don't print lines, just count them
-C	Context; print some lines above and below each line that matches (not implemented in this version; left as an exercise for the reader)
`-f` *pattern*	Take pattern from file named after `-f` instead of from command line
-h	Suppress printing filename ahead of lines
`-i`	Ignore case
-l	List filenames only: don't print lines, just the names they're found in
`-n`	Print line numbers before matching lines
-r	Recursive mode (also allowed at `-R`)
`-s`	Suppress printing certain error messages
-v	Invert: print only lines that do NOT match the pattern

The Unix world features several getopt library routines for parsing command-line arguments, so I have a reimplementation of this in Java. As usual, because `main()` runs in a static context but our application main line does not, we could wind up passing a lot of information into the constructor. To save space, this version just uses global variables to track the settings from the command line. Unlike the Unix `grep` tool, this one does not yet handle combined options, so `-l -r -i` is OK, but `-lri` will fail, due to a limitation in the `GetOpt` parser used.

The program basically just reads lines, matches the pattern in them, and, if a match is found (or not found, with `-v`), prints the line (and optionally some other stuff, too). To save space, the code is not shown here, as it largely combines techniques shown previously. It remains available in *darwinsys-api/src/main/java/com/darwinsys/regex/JGrep.java* or online at my GitHub repository.

1 We're starting to see contexts where the old form isn't accepted (e.g., as a field in a classless main).

2 Non-Unix fans fear not, for you can use tools like `grep` on Windows systems using one of several packages. One is an open source package called git bash, which includes `git` and several tools including `grep`. Another is Microsoft's `findstr /R` command for Windows. Or you can use my JGrep program in Recipe 4.10 if you don't have `grep` on your system. Incidentally, the name `grep` comes from an ancient Unix line editor command `g/RE/p`, the command to find the regex globally in all lines in the edit buffer and print the lines that match—just what the `grep` program does to lines in files.

3 REDemo was inspired by (but does not use any code from) a similar program provided with the now-retired Apache Jakarta Regular Expressions package.

4 Or a few related Unicode characters, including the next-line (`\u0085`), line-separator (`\u2028`), and paragraph-separator (`\u2029`) characters.

Previous chapter
< 3. Strings and Things

Next chapter
5. Numbers >